

```
In [47]: l=[7,8,4,9]
print(l)
```

[7, 8, 4, 9]

```
In [48]: import numpy as np
```

```
In [49]: import numpy as np
a=np.array([1,2,3])
print(a)
```

[1 2 3]

basic

```
In [50]: a=np.array([1, 2, 3])
b=np.array([1, 2, 3])
c=a + b
d=a-b
e=a*b
f=a/b
g=a%6
h=a**b
print(c)
print(d)
print(e)
print(f)
print(g)
print(h)
```

[2 4 6]
[0 0 0]
[1 4 9]
[1. 1. 1.]
[1 2 3]
[1 4 27]

```
In [51]: a=[1,2,3]
b=[1,2,3]
c=[]
for i in range(len(a)):
    c.append(a[i]+b[i])
print(c)
```

[2, 4, 6]

dimension

```
In [52]: a=np.array([[1,2,3],[3,2,1],[1,3,2]])
print(a)
```

[[1 2 3]
 [3 2 1]
 [1 3 2]]

```
In [53]: print(a.shape)
print(a.ndim)
```

```
print(a.dtype)
print(a.size)
```

```
(3, 3)
2
int32
9
```

creation

In [54]: `import numpy as np`

```
print("1d:", np.ones(3))
print("2d:", np.ones((1, 3)))

print("1d:", np.zeros(3))
print("2d:", np.zeros((1, 3)))
```

```
1d: [1. 1. 1.]
2d: [[1. 1. 1.]]
1d: [0. 0. 0.]
2d: [[0. 0. 0.]]
```

In [55]: `import numpy as np`

```
print(np.arange(10))
print(np.linspace(0,20,5))
```

```
[0 1 2 3 4 5 6 7 8 9]
[ 0.  5. 10. 15. 20.]
```

In [56]: `a=np.random.randn(1,100,50)`
`b=np.random.randint(1,50,25)`
`print(a)`
`print(b)`

```
[[[-0.6398766  0.03546528  0.16351887 ...  0.1057752 -1.60194094
   -1.99035626]
 [ 0.25817357 -0.60103027 -1.18193519 ...  0.47753473  0.57704021
   -1.62818014]
 [ 0.41389842 -0.5150833  1.26499835 ...  0.48018241 -1.21787413
    0.75742584]
 ...
 [ 0.12383998  0.91033156  1.15327648 ... -0.50628019 -1.68600808
    0.65951781]
 [ 0.92861377  0.53930862 -1.88244416 ...  0.37273812 -0.33685813
   -0.34184155]
 [-1.40769522  0.77039462 -0.19621921 ...  1.56741019  0.15580751
   -1.4622278 ]]]
[34 18 27 46 24  4 37  5  6 26 32 42 12 46 12 49 37 45 11 42  9  5 16 40
 26]
```

In [57]: `a=np.random.rand(1,100,50)`
`print(a)`

```
[[[0.58337681 0.75341588 0.54284327 ... 0.66681017 0.44957626 0.33160397]
 [0.22344429 0.10071401 0.11979452 ... 0.37685233 0.73217479 0.94520478]
 [0.52369611 0.49026969 0.54609392 ... 0.20172428 0.45211155 0.01130995]
 ...
 [0.9178499 0.03624448 0.51029551 ... 0.13373034 0.2184048 0.65282703]
 [0.75743189 0.109176 0.15704399 ... 0.23057614 0.10838698 0.20656114]
 [0.31458924 0.06260391 0.66676863 ... 0.27268046 0.66210227 0.4651924
 4]]]
```

index and slicing

```
In [58]: a=np.random.randint(1,10,10)
print(a)
print("index",a[-1])
print("slicing",a[1:4:1])
print("slicing",a[9:5:-1])
```

```
[8 8 4 8 8 9 6 2 7 9]
index 9
slicing [8 4 8]
slicing [9 7 2 6]
```

```
In [59]: arr = np.array([[1, 2, 3],[4, 5, 6],[7, 8, 9]])
print("index:",arr[0, 1])
print("index:",arr[2, 2])
print("slicing:",arr[0:2, :])
```

```
index: 2
index: 9
slicing: [[1 2 3]
 [4 5 6]]
```

```
In [60]: a = np.array([np.random.randint(1, 10, 5),np.random.randint(11, 20, 5)])
print(a)
print(a[:,6:9])
```

```
[[ 1  1  3  2  2]
 [15 19 16 19 14]]
[]
```

boolean

```
In [61]: a=np.random.randint(1,5,5)
print(a)
print(a>3)
print(a[a%2==0])
```

```
[2 3 1 3 4]
[False False False False  True]
[2 4]
```

Logical Comparison& ArithmeticOperations

```
In [62]: import numpy as np

a = np.array([10, 20, 30])
b = np.array([1, 2, 3])
a+=10
b-=5
print(a + b)
```

[16 27 38]

```
In [63]: arr = np.array([10, 20, 30])

print(arr > 15)
print(arr == 20)
print(arr != 10)
```

```
[False True True]
[False True False]
[False True True]
```

```
In [64]: arr = np.array([10, 20, 30, 40])

print((arr > 15) & (arr < 35))
print((arr < 15) | (arr > 35))
print(~(arr > 20))
```

```
[False True True False]
[ True False False  True]
[ True  True False False]
```

```
In [65]: import numpy as np

a = np.array([[1, 2],
              [3, 4]])

b = np.array([[5, 6],
              [7, 8]])

print(a + b)
print(a - b)
print(a * b)
print(a / b)
```

```
[[ 6  8]
 [10 12]]
[[-4 -4]
 [-4 -4]]
[[ 5 12]
 [21 32]]
[[0.2      0.33333333]
 [0.42857143 0.5      ]]
```

```
In [66]: a = np.array([[1, 5],
                       [7, 3]])

print(a > 3)
```

```
[[False  True]
 [ True False]]
```

```
In [67]: a = np.array([[1, 2],
                       [3, 4]])

print((a > 1) & (a < 4))
```

```
[[False  True]
 [ True False]]
```

mathematical operations are Element-wise Operations,Broadcasting

```
In [68]: import numpy as np

a = np.array([[1, 2],
              [3, 4]])

b = np.array([[5, 6],
              [7, 8]])

print(a + b)
print(a * b)
```

[[6 8]
 [10 12]]
[[5 12]
 [21 32]]

```
In [69]: a = np.array([[1, 2],
                      [3, 4]])

b = np.array([10, 20])

print(a + b)
```

[[11 22]
 [13 24]]

```
In [70]: a = np.array([[1, 2],
                      [3, 4]])

b = np.array([[5, 6],
              [7, 8]])

print(np.dot(a, b))
print(a.T)
```

[[19 22]
 [43 50]]
[[1 3]
 [2 4]]

universa operation

```
In [71]: print(np.sqrt(a))
print(np.abs(a))
print(np.log(a))
print(np.exp(a))
print(np.round(a))
print(np.ceil(a))
```

```
[[1.          1.41421356]
 [1.73205081  2.          ]]
[[1 2]
 [3 4]]
[[0.          0.69314718]
 [1.09861229  1.38629436]]
[[ 2.71828183  7.3890561 ]
 [20.08553692 54.59815003]]
[[1 2]
 [3 4]]
[[1. 2.]
 [3. 4.]]
```

statictical

```
In [72]: print("mean",np.mean(a))
print("median",np.median(a))
print("std",np.std(a))
print("var0",np.var(a))
print("max",np.max(a))
print("min",np.min(a))
print("percent",np.percentile(a,100))
print("argmin",np.argmin(a))
print("argmax",np.argmax(a))
b=np.array([1,2,3,45])
c=np.array([2,4,5,6])
print("cor",np.corrcoef(b,c))
```

```
mean 2.5
median 2.5
std 1.118033988749895
var0 1.25
max 4
min 1
percent 4.0
argmin 0
argmax 3
cor [[1.          0.70985277]
 [0.70985277  1.          ]]
```

create 2d numpy array of student scores ;compute average,high,normalize

```
In [75]: import numpy as np
from sklearn.preprocessing import MinMaxScaler
mark=np.random.randint(1,100,size=(5,5))
print("mark",mark)
print("average",round(np.mean(np.sum(mark,axis=0))))
print("highesh",np.max(np.sum(mark,axis=0)))
s=MinMaxScaler()
nor=s.fit_transform(mark)
print("normalize",nor)
```

```

mark [[15 92 63 11 46]
      [ 5 25 88  3 75]
      [69 14 18 22 20]
      [21 70 67 15 83]
      [46 30 45 57 68]]
average 214
highest 292
normalize [[0.15625      1.          0.64285714 0.14814815 0.41269841]
           [0.          0.14102564 1.          0.          0.87301587]
           [1.          0.          0.          0.35185185 0.          ]
           [0.25        0.71794872 0.7         0.22222222 1.          ]
           [0.640625    0.20512821 0.38571429 1.          0.76190476]]

```

```

In [74]: n = (mark - np.min(mark)) / (np.max(mark) - np.min(mark))
print(n)

```

```

[[0.91578947 0.87368421 0.87368421 0.54736842 1.          ]
 [0.44210526 0.67368421 0.24210526 0.65263158 0.66315789]
 [0.03157895 0.04210526 0.81052632 0.98947368 0.          ]
 [0.92631579 0.62105263 0.63157895 0.46315789 0.30526316]
 [0.65263158 0.94736842 0.32631579 0.36842105 0.22105263]]

```

given arr=np.array([10,20,30,40,50]),write numpy expressino to get all element > 25 and compute sum

```

In [76]: import numpy as np
arr = np.array([10, 20, 30, 40, 50])
print(arr)
print(arr[arr > 25].sum())

```

```

[10 20 30 40 50]
120

```

```

In [77]: a=np.random.randint(1,10,size=(3,1))
b=np.random.randint(1,10,size=(1,4))
print(a+b)

```

```

[[ 4 10  9  8]
 [ 5 11 10  9]
 [ 7 13 12 11]]

```